

```

import java.util.*;

public class productSegmentTree {
    static int tree[], n, arr[];

    public static void build(int idx, int s, int e) {
        if (s == e) {
            tree[idx] = (arr[s] == 0) ? 1 : 0;
            return;
        }
        int mid = (s + e) / 2;
        int l = idx * 2 + 1;
        int r = idx * 2 + 2;
        build(l, s, mid);
        build(r, mid + 1, e);

        tree[idx] = tree[l] + tree[r];
    }

    public static void update(int idx, int s, int e, int pos, int val) {
        if (s == e) {
            tree[idx] = (val == 0) ? 1 : 0;
            arr[s] = val;
            return;
        }

        int mid = (s + e) / 2;
        int l = idx * 2 + 1;
        int r = idx * 2 + 2;

        if (pos <= mid) update(l, s, mid, pos, val);
        else update(r, mid + 1, e, pos, val);

        // Recompute the sum of zeros for the parent node
        tree[idx] = tree[l] + tree[r];
    }

    public static int query(int idx, int s, int e, int l, int r) {
        if (s > r || e < l) {
            return 0;
        }

        if (s >= l && e <= r) {
            return tree[idx];
        }
    }
}

```

```

int mid = (s + e) / 2;

int left = query(idx * 2 + 1, s, mid, l, r);
int right = query(idx * 2 + 2, mid + 1, e, l, r);

return left + right;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    if (!sc.hasNextInt()) return;

    n = sc.nextInt();
    arr = new int[n];
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }

    tree = new int[4 * n];
    build(0, 0, n - 1);

    int q = sc.nextInt();
    while (q-- > 0) {
        int type = sc.nextInt();
        if (type == 1) {
            int pos = sc.nextInt();
            int val = sc.nextInt();
            update(0, 0, n - 1, pos, val);
        } else {
            int l = sc.nextInt();
            int r = sc.nextInt();
            System.out.println(query(0, 0, n - 1, l, r));
        }
    }
    sc.close();
}
}

```